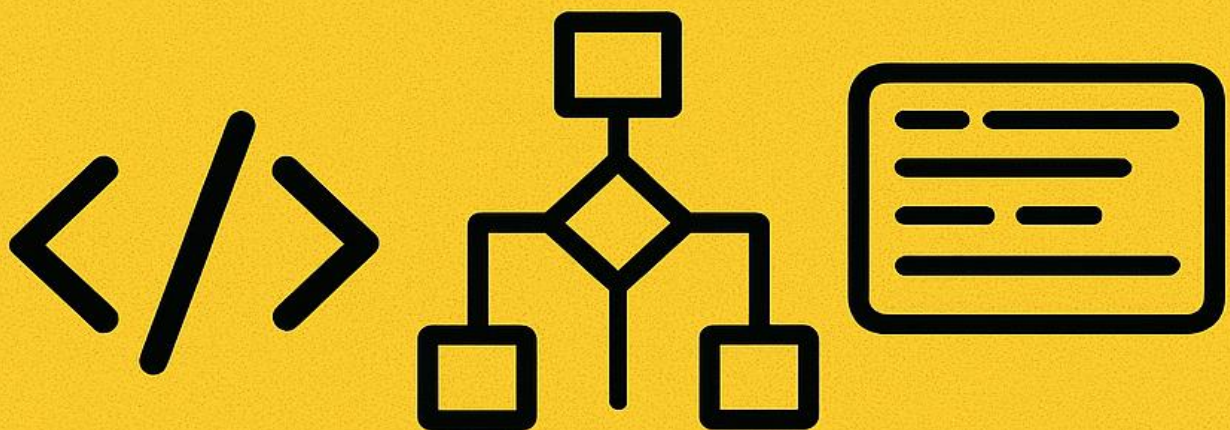


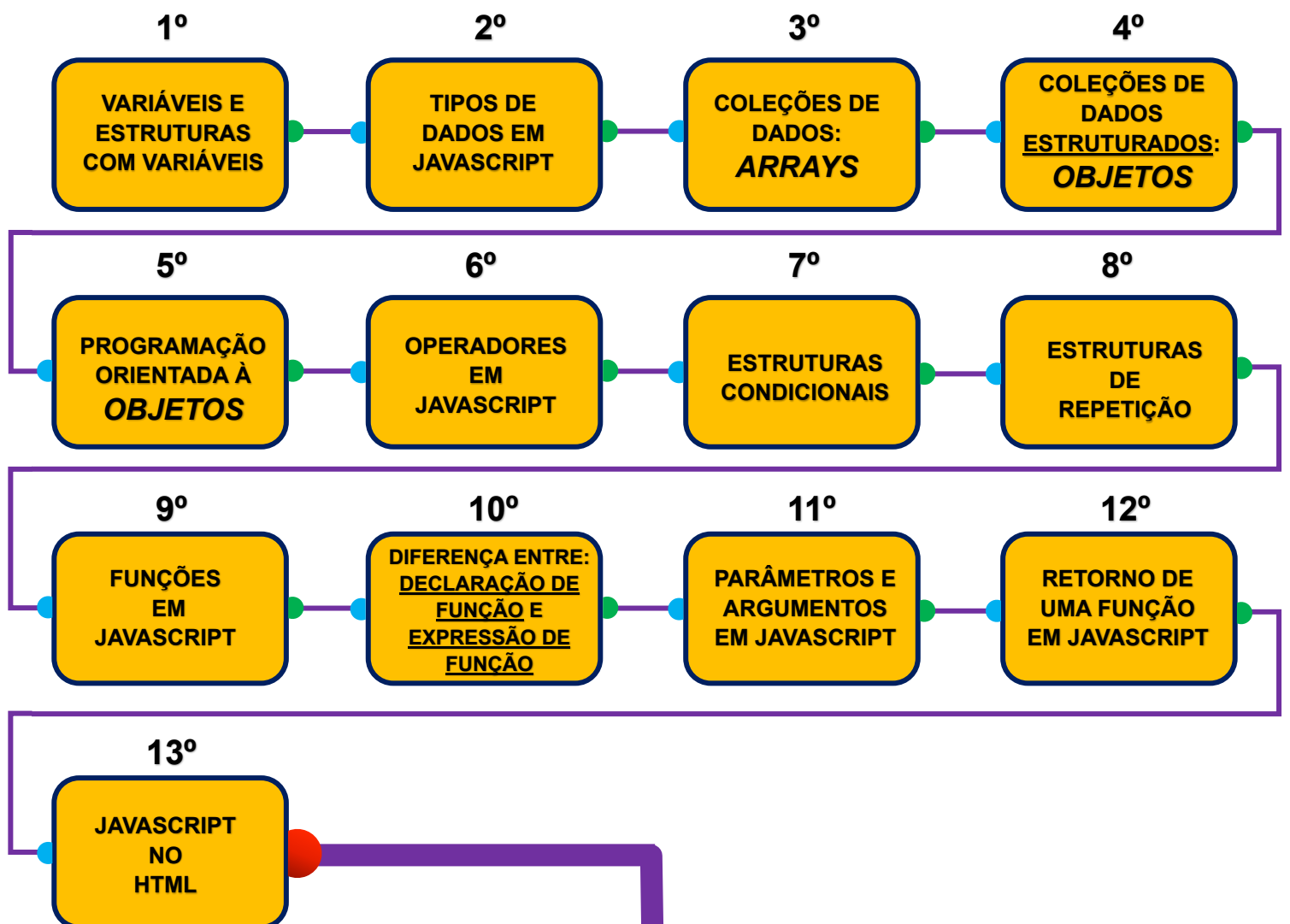
FUNDAMENTOS DA PROGRAMAÇÃO E LÓGICA



FASE I

ROTEIRO DE VIAGEM **JAVASCRIPT**

FASE I – FUNDAMENTOS DA PROGRAMAÇÃO E LÓGICA



AVANTE!

VARIÁVEIS E ESTRUTURAS COM VARIÁVEIS CAIXINHAS DE MEMÓRIA

1. Formas de declarar variáveis (criar "caixinhas" de memória)

São as palavras reservadas que você usa para criar variáveis em JS:

var nome; → forma antiga (não recomendado hoje em dia).

let nome; → forma moderna, permite mudar o valor depois.

const nome; → forma moderna, mas o valor não pode mudar (constante).

Exemplo:

```
var idade = 27;
```

```
let cidade = "Recife";
```

```
const pi = 3.14159;
```

2. Estruturas (tipos de valores) que você pode guardar na variável

Quando você faz `let x = ...`, o `...` pode ser qualquer tipo de dado.

Por exemplo:

- Array (lista): `[]`
- `let numeros = [1, 2, 3];`

É uma coleção de valores.

- **Matriz (array dentro de array): `[[ ], [ ]]`**

- `let matriz = [ [ 1, 2 ], [ 3, 4 ] ];`

É como uma tabela (linhas e colunas).

- Objeto: `{ }`
- `let pessoa = { nome: "Ana", idade: 19 };`

É uma estrutura com pares chave → valor.

3. Resumindo

👉 `var`, `let`, `const` → são **formas de declarar variáveis**.

👉 `[]`, `[[ ]]`, `[[ ]]`, `{ }` → são **tipos de valores** (estruturas de dados) que você pode colocar dentro de uma variável.

TIPOS DE DADOS EM JAVASCRIPT

TRATA DOS TIPOS DE DADOS QUE
PODEMOS “GUARDAR”
NAS VARIÁVEIS EM JAVASCRIPT.

Tipos Primitivos (são imutáveis e armazenam valores simples)

1. String → Texto

```
let nome = "Robson";  
let saudacao = 'Olá!';  
let frase = `Meu nome é ${nome}`;
```

2. Number → Números inteiros e decimais

```
let idade = 27;  
let preco = 19.99;  
let infinito = Infinity;  
let notANumber = NaN; // resultado de cálculos inválidos
```

3. BigInt → Números inteiros muito grandes

```
let numeroGrande = 123456789012345678901234567890n;
```

4. Boolean → Verdadeiro ou falso

```
let aprovado = true;  
let reprovado = false;
```

5. Undefined → Variável declarada mas sem valor atribuído

```
let x;  
console.log(x); // undefined
```

6. Null → Representa ausência intencional de valor

```
let y = null;
```

7. Symbol → Valores únicos e imutáveis (usado em identificadores de objetos)

O **Symbol** é um tipo de dado **primitivo** do JavaScript, introduzido no **ES6 (2015)**. Ele serve para criar **identificadores únicos**.

```
let id = Symbol("id");
```

Mesmo que dois símbolos tenham a mesma descrição, eles **não são iguais**.

```
let s1 = Symbol("id");
```

```
let s2 = Symbol("id");
```

```
console.log(s1 === s2); // false
```

◆ Tipos de Referência (Objetos)

São estruturas mais complexas, armazenadas em referência.

1. Object → Estrutura de chave-valor

```
let pessoa = {  
  nome: "Ana",  
  idade: 19  
};
```

2. Array → Lista ordenada de valores

```
let cores = ["vermelho", "verde", "azul"];
```

3. Function → Função também é um tipo de dado em JS

```
function soma(a, b) {  
  return a + b;  
}
```

4. Date → Datas

```
let hoje = new Date();
```

5. Outros objetos nativos → Map, Set, WeakMap, WeakSet, RegExp, Error etc.

◆ Resumindo

- **Primitivos:** string, number, bigint, boolean, undefined, null, symbol
- **Referência (objetos):** object, array, function, date, map, set, etc.

COLEÇÕES DE DADOS

ARRAYS

O que é um array em JavaScript?

Um array é uma lista ordenada de valores, que podem ser de qualquer tipo (números, strings, objetos, até outros arrays).

👉 Ele serve para armazenar vários dados em uma única variável.

Exemplo:

```
let frutas = ["maçã", "banana", "uva"];
```

Aqui, `frutas[0]` = "maçã", `frutas[1]` = "banana", `frutas[2]` = "uva".

Os índices começam do 0.

◆ Formas de criar um array

1. Usando colchetes [] (mais comum):

```
let numeros = [1, 2, 3, 4, 5];
```

2. Usando o construtor `new Array( )`:

```
let cores = new Array("vermelho", "verde", "azul");
```

3. Array vazio e depois adicionar elementos:

```
let lista = [ ];
```

```
lista[0] = "primeiro";
```

```
lista[1] = "segundo";
```

◆ Principais métodos e formas de manipular arrays

➡ Adicionar e remover elementos

- **push()** → adiciona no final

```
let frutas = ["maçã", "banana"];
```

```
frutas.push("laranja"); // ["maçã", "banana", "laranja"]
```

- **pop()** → remove o último

```
frutas.pop( ); // ["maçã", "banana"]
```

- **unshift()** → adiciona no início

```
frutas.unshift("uva"); // ["uva", "maçã", "banana"]
```

- **shift()** → remove do início

```
frutas.shift( ); // ["maçã", "banana"]
```

➔ Acessar e modificar

- **frutas[index]** → acessa pelo índice

```
console.log(frutas[1]); // "banana"
```

- **Modificar:**

```
frutas[1] = "manga"; // ["maçã", "manga"]
```

➔ Procurar dentro do array

- **indexOf(valor)** → retorna a posição

```
frutas.indexOf("manga"); // 1
```

- **includes(valor)** → verifica se existe

```
frutas.includes("maçã"); // true
```

➔ Juntar, cortar e copiar

- **concat()** → junta arrays

```
let a = [1,2];
```

```
let b = [3,4];
```

```
let c = a.concat(b); // [1,2,3,4]
```

- **slice(início, fim)** → copia parte do array

```
let numeros = [1,2,3,4,5];
```

```
numeros.slice(1,3); // [2,3]
```

- **splice(início, quantos, itens...)** → remove ou substitui

```
numeros.splice(2,1,"dez"); // remove 1 item no índice 2 e adiciona "dez"
```

➔ Transformar

- **join()** → array em string

```
frutas.join(" - "); // "maçã - manga"
```

- **toString()** → parecido com join

```
frutas.toString( ); // "maçã,manga"
```

➔ Iterar (percorrer)

- **for clássico**

```
for (let i = 0; i < frutas.length; i++) {
```

```
  console.log(frutas[i]);
```

```
}
```

- **for...of**

```
for (let fruta of frutas) {
  console.log(fruta);
}
```

- **forEach()**

```
frutas.forEach(f => console.log(f));
```

➡ Métodos modernos de manipulação

- **map()** → cria um novo array transformando os itens

```
let numeros = [1,2,3];
let dobrados = numeros.map(n => n * 2); // [2,4,6]
```

- **filter()** → cria novo array com base em uma condição

```
let pares = numeros.filter(n => n % 2 === 0); // [2]
```

- **reduce ()** → reduz a um único valor

```
let soma = numeros.reduce((acc, n) => acc + n, 0); // 6
```

- **find()** → encontra o primeiro que satisfaz a condição

```
let maiorQue2 = numeros.find(n => n > 2); // 3
```

- **some()** → retorna true se algum satisfaz

```
numeros.some(n => n > 2); // true
```

- **every()** → retorna true se todos satisfazem

```
numeros.every(n => n > 0); // true
```

- **sort()** → ordena

```
let nums = [10,1,5];
nums.sort((a,b) => a-b); // [1,5,10]
```

🔥 Resumo rápido:

- **Arrays = listas de valores.**
- **Criar com [] ou new Array().**
- **Métodos principais: push, pop, shift, unshift, splice, slice, concat.**
- **Iterar com for, forEach, map.**
- **Buscar com indexOf, includes, find.**
- **Transformar com map, filter, reduce.**

COLEÇÃO DE DADOS
ESTRUTURADOS
“OBJETOS”

O que é um objeto em JavaScript?

Um objeto é como uma caixa com várias gavetas onde cada gaveta tem um nome (chave) e dentro dela você guarda um valor.

Esses valores podem ser números, textos, arrays, funções, até outros objetos.

Exemplo simples:

```
let pessoa = {  
  nome: "Ana",  
  idade: 25,  
  profissao: "Engenheira"  
};
```

Aqui, o objeto pessoa tem propriedades:

- chave nome com valor "Ana"
- chave idade com valor 25
- chave profissao com valor "Engenheira"



É como um "fichário" que organiza dados relacionados.

◆ Formas de criar objetos

1. Objeto literal (mais usado):

```
let carro = {  
  marca: "Toyota",  
  modelo: "Corolla",  
  ano: 2022  
};
```

2. Com new Object():

```
let pessoa = new Object( );  
pessoa.nome = "Diego";  
pessoa.idade = 27;
```

3. Com função construtora:

```
function Pessoa(nome, idade) {  
  this.nome = nome;  
  this.idade = idade;  
}  
  
let p1 = new Pessoa("Ana", 19);  
let p2 = new Pessoa("Carlos", 30);
```

4. Com class (forma moderna de criar objetos a partir de “moldes”):

```
class Pessoa {  
  constructor(nome, idade) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
}  
  
let pessoa1 = new Pessoa("João", 40);
```

◆ Como manipular dados em um objeto

➡ Acessar propriedades

- Notação ponto “ . ”

```
console.log(pessoa.nome);
```

- Notação colchete:

```
console.log(pessoa["idade"]);
```

➡ Adicionar ou modificar

```
pessoa.profissao = "Professor"; // adiciona
```

```
pessoa.idade = 28; // modifica
```

➡ Remover

```
delete pessoa.profissao;
```

➡ Percorrer propriedades

- for...in:

```
for (let chave in pessoa) {  
  console.log(chave, pessoa[chave]);  
}
```

- **Object.keys(obj)** → pega só as chaves
- **Object.values(obj)** → pega só os valores
- **Object.entries(obj)** → pega pares [chave, valor]

PROGRAMAÇÃO ORIENTADA A OBJETOS

ESTÁ RELACIONADO AS FORMAS QUE
PODEMOS CRIAR UM OU MÚLTIPLOS OBJETOS
USANDO UM RECURSO CHAMADO “CLASSE”
E AS FORMAS QUE PODEMOS MANIPULÁ-LOS E ADICIONARMOS
Features “CARACTERÍSTICAS”, COMO:
HERANÇA, POLIMORFISMO, ENCAPSULAMENTO.

POO em JavaScript (com analogias)

Agora que você entendeu objetos, vamos subir de nível.

1. Classe

👉 Pense em classe como uma fôrma de bolo.

A fôrma (classe) não é o bolo em si, mas você pode usar ela para criar vários bolos (objetos).

Exemplo:

```
class Animal {  
  constructor(nome, tipo) {  
    this.nome = nome;  
    this.tipo = tipo;  
  }  
  emitirSom( ) {  
    console.log(`${this.nome} fez um som!`);  
  }  
}
```

Aqui Animal é a **fôrma**, e this.nome / this.tipo são as “**características**” de cada animal.

2. Objeto (instância da classe)

👉 O bolo feito pela fôrma. Cada objeto é um “exemplar único”.

```
let cachorro = new Animal("Rex", "Cachorro");
```

```
let gato = new Animal("Mimi", "Gato");
```

3. Herança

👉 Imagine que você tem uma classe mãe chamada Animal, e dela você cria classes filhas Cachorro e Gato.

As filhas herdam características da mãe, mas podem ter coisas próprias também.

```
class Cachorro extends Animal {  
  emitirSom( ) {  
    console.log("Au au!");  
  }  
}
```

```
class Gato extends Animal {  
  emitirSom() {  
    console.log("Miau!");  
  }  
}
```

Aqui, Cachorro e Gato herdam de Animal, mas sobrescrevem o método emitirSom().

4. Polimorfismo

👉 "Polimorfismo" significa "muitas formas".

Ou seja, diferentes objetos podem responder de formas diferentes ao mesmo comando.

Exemplo:

```
let animais = [new Cachorro("Rex"), new Gato("Mimi")];
```

```
for (let a of animais) {  
  a.emitirSom(); // cada um faz seu som específico  
}
```

O mesmo método emitirSom() tem resultados diferentes dependendo do objeto.

5. Encapsulamento

👉 **Pense em um carro: você só dirige (usa o volante, acelerador), mas não vê a complexidade do motor por dentro.**

No JS, usamos isso para esconder detalhes internos e expor apenas o necessário.

```
class ContaBancaria {  
  #saldo = 0; // propriedade privada  
  
  depositar(valor) {  
    this.#saldo += valor;  
  }  
  
  verSaldo( ) {  
    return this.#saldo;  
  }  
}
```

```
let conta = new ContaBancaria();  
conta.depositar(100);  
console.log(conta.verSaldo()); // 100  
// console.log(conta.#saldo); // ERRO → saldo é privado
```

◆ Métodos úteis de objetos

- **Object.keys(obj)** → retorna array com chaves
- **Object.values(obj)** → retorna array com valores
- **Object.entries(obj)** → retorna pares [chave, valor]
- **Object.assign(destino, origem)** → copia propriedades
- **JSON.stringify(obj)** → transforma objeto em texto (string JSON)
- **JSON.parse(string)** → transforma texto em objeto

📌 Resumindo com analogia:

- **Classe** = fôrma do bolo.
- **Objeto** = o bolo feito a partir da fôrma.
- **Herança** = bolo de chocolate e bolo de morango feitos da mesma fôrma, mas com ingredientes extras.
- **Polimorfismo** = cada bolo tem sabor diferente, mas todos são “bolo”.
- **Encapsulamento** = você come o bolo, mas não vê a receita secreta por trás.

OPERADORES EM JAVASCRIPT

SÃO SÍMBOLOS / SINAIS QUE

SERVEM PARA REALIZAR VÁRIAS OPERAÇÕES ENTRE VARIÁVEIS

DENTRO DE DIFERENTES CONTEXTOS, COMO,

OPERAÇÕES MATEMÁTICAS, COMPARAÇÕES,

ATRIBUIÇÃO DE VALORES, ENTRE OUTROS.

Operadores são símbolos que o JavaScript usa para fazer contas, comparações e operações lógicas.

Vou dividir por categorias pra ficar claro:

◆ 1. Operadores Aritméticos (contas matemáticas)

+ // soma
- // subtração
* // multiplicação
/ // divisão
% // resto da divisão (módulo)
** // potência (ex: $2^{**}3 = 8$)

✅ Exemplo:

```
let a = 10, b = 3;  
console.log(a + b); // 13  
console.log(a % b); // 1 (resto)  
console.log(a ** b); // 1000 ( $10^3$ )
```

2. Operadores de Atribuição (guardar valor em variável)

= // atribuição simples
+= // soma e atribui
-= // subtrai e atribui
*= // multiplica e atribui
/= // divide e atribui
%= // resto e atribui

✅ Exemplo:

```
let x = 5;  
x += 3; //  $x = x + 3 \rightarrow 8$   
x *= 2; //  $x = x * 2 \rightarrow 16$ 
```

◆ 3. Operadores de Comparação (true/false)

`==` // igual (compara só valor, ignora tipo)
`===` // estritamente igual (compara valor e tipo)
`!=` // diferente (ignora tipo)
`!==` // estritamente diferente (valor e tipo)
`>` // maior que
`<` // menor que
`>=` // maior ou igual
`<=` // menor ou igual

✅ Exemplo:

```
console.log(5 == "5"); // true (só valor)
console.log(5 === "5"); // false (valor igual, mas tipo diferente)
console.log(10 > 5); // true
```

◆ 4. Operadores Lógicos

`&&` // E (true se as duas condições forem verdadeiras)
`||` // OU (true se pelo menos uma condição for verdadeira)
`!` // NÃO (inverte true/false)

✅ Exemplo:

```
let idade = 20;
console.log(idade > 18 && idade < 30); // true
console.log(idade < 18 || idade > 60); // false
console.log(!(idade > 18)); // false
```

◆ 5. Operadores de Incremento/Decremento

`++` // adiciona 1
`--` // subtrai 1

✅ Exemplo:

```
let n = 10;
n++; // 11
n--; // 10
```

◆ 6. Operador Condicional (Ternário)

condição ? valorSeVerdadeiro : valorSeFalso

✅ Exemplo:

```
let idade = 17;  
let podeDirigir = idade >= 18 ? "Sim" : "Não";  
console.log(podeDirigir); // "Não"
```

◆ 7. Operadores de Tipo

typeof // mostra o tipo de um valor

instanceof // verifica se um objeto é de uma classe

✅ Exemplo:

```
console.log(typeof 42); // "number"  
console.log(typeof "Olá"); // "string"  
console.log([1,2,3] instanceof Array); // true
```

◆ 8. Operador de Encadeamento Opcional (ES2020+)

?. // evita erro se a propriedade não existir

✅ Exemplo:

```
let pessoa = { nome: "Ana" };  
console.log(pessoa?.idade); // undefined (não dá erro)
```

🔗 Resumo prático:

- **Aritméticos:** + - * / % **
- **Atribuição:** = += -= *= /= %=
- **Comparação:** == === != !== > < >= <=
- **Lógicos:** && || !
- **Incremento:** ++ --
- **Ternário:** ? :
- **Tipo:** typeof, instanceof
- **Encadeamento:** ?.

ESTRUTURAS CONDICIONAIS

**SÃO BLOCOS DE CÓDIGO
QUE RECEBEM UMA CONDIÇÃO
E EXECUTAM DETERMINADA TAREFA
CASO ESSA CONDIÇÃO SEJA VERDADEIRA OU FALSA.**

As estruturas condicionais em JavaScript são usadas para executar blocos de código diferentes dependendo de uma condição (verdadeira ou falsa).

Aqui estão as principais:

1. if

Executa um bloco de código se a condição for verdadeira.

```
let idade = 18;
```

```
if (idade >= 18) {  
    console.log("Você é maior de idade.");  
}
```

2. if...else

Permite executar um bloco se a condição for verdadeira e outro se for falsa.

```
let idade = 16;
```

```
if (idade >= 18) {  
    console.log("Você é maior de idade.");  
} else {  
    console.log("Você é menor de idade.");  
}
```

3. if...else if...else

Permite testar várias condições em sequência.

```
let nota = 75;
```

```
if (nota >= 90) {  
    console.log("Aprovado com mérito!");  
} else if (nota >= 60) {  
    console.log("Aprovado!");  
} else {  
    console.log("Reprovado.");  
}
```

4. switch

Útil quando se deseja comparar a mesma variável com muitos valores diferentes.

```
let cor = "verde";
```

```
switch (cor) {  
  case "vermelho":  
    console.log("Pare!");  
    break;  
  case "amarelo":  
    console.log("Atenção!");  
    break;  
  case "verde":  
    console.log("Siga!");  
    break;  
  default:  
    console.log("Cor inválida.");  
}
```

5. Operador ternário (? :)

Uma forma mais curta de escrever um if...else simples.

```
let idade = 20;
```

```
let resultado = idade >= 18 ? "Maior de idade" : "Menor de idade";  
console.log(resultado);
```

6. Operadores lógicos em condições

Você pode combinar várias condições:

```
let idade = 22;
```

```
let temCarteira = true;
```

```
if (idade >= 18 && temCarteira) {  
  console.log("Pode dirigir.");  
}
```

```
if (idade < 18 || ! temCarteira) {  
  console.log("Não pode dirigir.");  
}
```

Resumindo:

- **if / else if / else** → para decisões com diferentes fluxos.
- **switch** → para várias opções baseadas em um único valor.
- **operador ternário** → para condições curtas em uma linha.

ESTRUTURAS DE REPETIÇÃO

SÃO BLOCOS DE CÓDIGO
QUE EXECUTAM INSTRUÇÕES MUITAS VEZES
CASO UMA CONDIÇÃO SEJA SATISFEITA.

As estruturas de repetição (loops) em JavaScript servem para executar um bloco de código várias vezes, até que uma condição seja satisfeita.

◆ Principais estruturas de repetição

1. for

Usado quando você sabe quantas vezes quer repetir.

```
for (let i = 0; i < 5; i++) {  
  console.log("Contagem:", i);  
}  
// Saída: 0, 1, 2, 3, 4
```

2. while

Executa o bloco enquanto a condição for verdadeira.

```
let i = 0;  
while (i < 5) {  
  console.log("Número:", i);  
  i++;  
}
```

3. do...while

Parecido com o while, mas executa pelo menos uma vez, mesmo que a condição seja falsa.

```
let i = 10;  
do {  
  console.log("Valor:", i);  
  i++;  
} while (i < 5);  
// Executa uma vez mesmo sendo falso
```

4. for...in

Percorre as propriedades (chaves) de um objeto.

```
let pessoa = { nome: "Ana", idade: 20 };  
  
for (let chave in pessoa) {
```

```
console.log(chave, "=", pessoa[chave]);  
}  
// Saída: nome = Ana, idade = 20
```

5. for...of

Percorre os valores de objetos iteráveis (arrays, strings, maps, sets...).

```
let cores = ["vermelho", "verde", "azul"];  
  
for (let cor of cores) {  
  console.log(cor);  
}  
// Saída: vermelho, verde, azul
```

◆ Palavras-chave importantes dentro de loops

1. break → Interrompe o loop imediatamente.

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;  
  console.log(i);  
}  
// Saída: 0,1,2,3,4
```

2. continue → Pula a iteração atual e vai para a próxima.

```
for (let i = 0; i < 5; i++) {  
  if (i === 2) continue;  
  console.log(i);  
}  
// Saída: 0,1,3,4
```

◆ Resumindo

- **for** → quando você sabe o número de repetições.
- **while** → quando não sabe exatamente quantas vezes vai repetir (condição aberta).
- **do...while** → garante pelo menos uma execução.
- **for...in** → percorre chaves de objetos.
- **for...of** → percorre valores de arrays/iteráveis.

FUNÇÕES EM JAVASCRIPT

SÃO BLOCOS DE CÓDIGOS
QUE ACEITAM PARÂMETROS (OPCIONAL)
PARA EXECUTAR INSTRUÇÕES (AÇÕES) ESPECÍFICAS.

FUNÇÕES JAVASCRIPT: Existem três formas de criar uma função.

1º Declaração de uma função: A função é declarada explicitamente.

```
function NomeDaFuncao (parâmetros de entrada) {  
    return de algo  
};
```

Chamada da função pelo nome e parênteses, exemplo:

```
NomeDaFuncao()
```

2º Expressão de uma função: A função é declarada (guardada) dentro de uma variável.

```
const NomeDaFuncao = function (parâmetros de entrada) {  
    return de algo  
};
```

Chamada da função pelo nome e parênteses, exemplo:

```
NomeDaFuncao();
```

Esse tipo de função recebe um nome, parâmetros de entrada - se houver - e, instruções de retorno. A função para ser executada deve ser chamada passando o nome dela.

3º Arrow Function:

```
const NomeDaFunção = (parâmetros) => {  
    return de instrução  
};
```

Arrow function é uma forma mais curta de criar uma função. A sintaxe dela é criando uma variável/constante com o nome da função e armazenando a função dessa variável/constante.

Para chamar uma Arrow Function segue o mesmo padrão da função normal:

```
NomeDaFunção()
```


OBSERVAÇÃO:

SEMPRE que for chamar uma função deve utilizar o “NomeDaFunção” seguido de parênteses “ () “. A função será executada totalmente.

Agora, caso queira somente o valor retornado da função, basta somente criar uma variável “LET ou CONST” e guardar o valor retornado da função apenas chamando-a pelo nome “NomeDaFunção” sem os parênteses.

O QUE SÃO MÉTODOS EM JAVASCRIPT?

Um **método** é, basicamente, **uma função que está dentro de um objeto**. No JavaScript, funções podem existir sozinhas ou dentro de objetos. Quando estão dentro de um objeto, chamamos de **método**.

Analogia:

Pense em um carro (objeto). Esse carro tem funções específicas (métodos), como:

carro.ligar()

carro.acelerar()

carro.frear()

O carro é o objeto, e essas ações que ele pode realizar são métodos.

QUANDO USAR UMA function expression, function declaration. OU UMA arrow function:

1. Função normal (function expression ou function declaration)

Você usa quando:

- ✓ Precisa de uma função que tenha seu próprio **this**, **arguments**, **super** ou **new.target**.
- ✓ Vai usar a função como método de um objeto.
- ✓ Vai usar a função como construtora (com new).

Exemplo:

// Método de objeto

```
const pessoa = {  
  nome: "Robson",  
  falar: function() {  
    console.log("Meu nome é " + this.nome);  
  }  
};  
  
pessoa.falar(); // Meu nome é Robson
```

Aqui, uma **arrow function** não funcionaria, porque ela não teria o `this` certo.

2. Arrow function

Você usa quando:

- ✓ Quer uma função **curta e simples**.
- ✓ Não precisa de `this` próprio (ela pega o `this` de fora).
- ✓ Vai usar em **callbacks** ou funções rápidas.
- ✓ Quer deixar o código mais limpo.

Exemplo:

// Callback com arrow

```
const numeros = [1, 2, 3, 4];
```

```
const dobrados = numeros.map(n => n * 2);
```

```
console.log(dobrados); // [2, 4, 6, 8]
```

✓ Resumindo fácil:

- **Função normal** → quando precisa de **this próprio**, **arguments**, ou **construtora**.
- **Arrow function** → quando quer **função curta**, **callback** ou **não precisa de this próprio**.

👉 Analogia rápida:

- **Função normal** é "independente", tem sua própria identidade (`this`, `arguments`).
- **Arrow function** é "colada no ambiente", só executa o que precisa sem criar nada novo.

DIFERENÇA ENTRE:

**Declaração de função e
Expressão de função
em JavaScript.**

DECLARAÇÃO DE FUNÇÃO (FUNCTION DECLARATION)

Uma declaração de função é uma forma de definir uma função com um nome explícito, que pode ser chamada em qualquer ponto do seu escopo devido ao hoisting. Ela descreve uma ação ou comportamento que pode ser reutilizado ao longo do programa.

Sintaxe:

```
function minhaFuncao(param) {  
    return param * 2;  
}
```

Características principais:

1. Hoisting (içamento):

A função é "elevada" para o topo do escopo, então você pode chamá-la antes de declará-la:

```
console.log(soma(2, 3)); // 5  
  
function soma(a, b) {  
    return a + b;  
}
```

2. Nome obrigatório:

Toda declaração precisa ter um nome (funções anônimas não podem ser declarações).

3. Uso comum:

Geralmente usada quando queremos funções reutilizáveis e claras no código.

Pontos importantes:

1. **Nome da função é obrigatório.**
2. **Pode ser chamada antes da definição** no código (por causa do hoisting).
3. **Usada para definir ações claras e reutilizáveis** no programa.
4. **Sempre cria uma referência no escopo em que foi definida**, normalmente o escopo global ou de uma função.

EXPRESSÃO DE FUNÇÃO (FUNCTION EXPRESSION)

Uma expressão de função é uma função que é atribuída a uma variável, constante ou propriedade de objeto. Diferente da declaração de função, ela não precisa ter um nome (pode ser anônima) e só pode ser chamada depois de definida, porque não sofre hoisting completo.

Sintaxe:

```
const minhaFuncao = function(param) {  
    return param * 2;  
};
```

Características principais:

1. Não sofre hoisting completo:

A variável `minhaFuncao` é elevada, mas só será definida quando o código chegar na linha da atribuição. Ou seja, não é possível chamar a função antes de definir a expressão:

```
console.log(soma(2, 3)); // Erro: soma não é função  
  
const soma = function(a, b) {  
    return a + b;  
};
```

2. Pode ser anônima ou nomeada:

- ✓ **Anônima:** `const soma = function(a, b) { return a + b; };`
- ✓ **Nomeada:** `const soma = function somaFunc(a, b) { return a + b; };`
O nome da função (se houver) só serve para referência interna.

3. Mais flexível:

Pode ser atribuída a variáveis, passada como argumento ou retornada de outras funções. É muito usada em callbacks ou funções como valores.

Resumindo:

Aspecto	Declaração de Função	Expressão de Função
Hoisting	Sim, pode ser chamada antes	Não totalmente, só a variável é içada
Nome	Obrigatório	Opcional (pode ser anônima)
Quando usar	Funções principais, reutilizáveis	Callbacks, funções temporárias, flexíveis
Exemplo	<code>function soma(a, b){ return a+b; }</code>	<code>const soma = function(a, b){ return a+b; };</code>

PARÂMETROS E ARGUMENTOS EM JAVASCRIPT

PARÂMETRO É A VARIÁVEL QUE UMA FUNÇÃO UTILIZA
ARGUMENTO É O VALOR PASSADO PARA ESTA VARIÁVEL (PARÂMETRO).

1º Parâmetros (Parameters)

Parâmetros são variáveis definidas na declaração da função que servem como "entradas" para a função. Eles são especificados dentro dos parênteses da função e recebem os valores quando a função é chamada.

Exemplo:

```
function somar(a, b) { // a e b são parâmetros
  return a + b;
}
```

- a e b são os parâmetros.
- Eles definem o que a função espera receber para executar sua tarefa.

2º Argumentos (Arguments)

Argumentos são os valores reais que você passa para a função quando a chama. Eles "preenchem" os parâmetros.

Exemplo:

```
console.log(somar(2, 3)); // 2 e 3 são argumentos
```

- 2 e 3 são os argumentos que serão usados pelos parâmetros a e b da função somar.

Resumo da diferença

Conceito	Onde aparece	O que representa
Parâmetro	Na definição da função	Variável que espera um valor
Argumento	Na chamada da função	Valor real passado para o parâmetro

Exemplo completo:

```
function cumprimentar(nome) { // 'nome' é o parâmetro
  console.log("Olá, " + nome + "!");
}
```

```
cumprimentar("Robson"); // "Robson" é o argumento
```



Saída: Olá, Robson!

RETORNO DE UMA FUNÇÃO

TODA FUNÇÃO RETORNA ALGO.

MESMO QUE NA FUNÇÃO NÃO SEJA DECLARADO O RETURN.

NESSE CASO, O RETORNO SERÁ “undefined”.

O que é o retorno de uma função?

Quando você chama uma função, ela pode executar instruções e, no final, retornar um valor usando a palavra-chave `return`.

Esse valor pode ser armazenado em uma variável, usado em cálculos, ou até passado para outra função.

◆ Exemplo básico

```
function somar(a, b) {  
  return a + b;  
}
```

```
let resultado = somar(5, 3);
```

```
console.log(resultado); // 8
```



Aqui, a função `somar` retorna a soma de `a + b`, e esse valor é guardado em `resultado`.

◆ Importante sobre `return`

1. Interrompe a função.

Tudo o que vem depois do `return` não é executado.

```
function teste() {  
  console.log("Antes do return");  
  return "Fim da função";  
  console.log("Depois do return"); // nunca será executado  
}
```

2. Pode retornar qualquer tipo de dado

```
function retornaTexto( ) { return "Olá"; }  
function retornaNumero( ) { return 42; }  
function retornaBooleano( ) { return true; }  
function retornaObjeto( ) { return { nome: "Ana", idade: 20 }; }  
function retornaArray( ) { return [1, 2, 3]; }
```

3. Funções sem return retornam undefined

```
function semRetorno() {  
  console.log("Executando...");  
}
```

```
let valor = semRetorno();  
console.log(valor); // undefined
```

4. Pode retornar outra função (conceito de função de ordem superior)

```
function multiplicador(fator) {  
  return function(numero) {  
    return numero * fator;  
  };  
}
```

```
let dobro = multiplicador(2);  
console.log(dobro(5)); // 10
```

◆ Diferença entre console.log e return

- **console.log** apenas exibe no console, não devolve nada.
- **return** devolve um valor para ser usado em outro lugar.

```
function exemplo() {  
  return 10;  
}
```

```
console.log(exemplo()); // mostra 10 no console  
let x = exemplo(); // x agora vale 10
```

👉 Resumindo:

O return é a forma de entregar um resultado da função para quem a chamou. Sem return, a função sempre retorna undefined.

JAVASCRIPT NO HTML

EXISTEM 3 FORMAS
DE INCLUIR JAVASCRIPT
EM UM DOCUMENTO HTML

1. JavaScript Interno (dentro do HTML)

O código JS fica dentro da tag `<script> ... </script>` no próprio arquivo HTML.

```
<!DOCTYPE html>

<html>

<head>
  <title>Exemplo</title>
</head>
<body>
  <h1>Olá!</h1>

  <script>
    alert("Este é um script interno!");
  </script>

</body>
</html>
```

◆ 2. JavaScript Externo (arquivo separado)

O código JS fica em um arquivo .js, e você o importa no HTML com a tag `<script src="...">`.

```
<!DOCTYPE html>

<html>

<head>
  <title>Exemplo</title>
</head>
<body>
  <h1>Exemplo com JS externo</h1>

  <script src="script.js"></script>

</body>
</html>
```

Arquivo script.js

alert("Este é um script externo!");

 Essa é a forma mais recomendada em projetos reais, pois separa HTML (estrutura) de JavaScript (lógica).

3. JavaScript Inline (dentro de atributos HTML)

O código JS é colocado diretamente em um atributo de evento de uma tag HTML, como onclick.

```
<!DOCTYPE html>

<html>

<head>

  <title>Exemplo</title>

</head>

<body>

  <button onclick="alert('Você clicou no botão!')">Clique aqui</button>

</body>

</html>
```

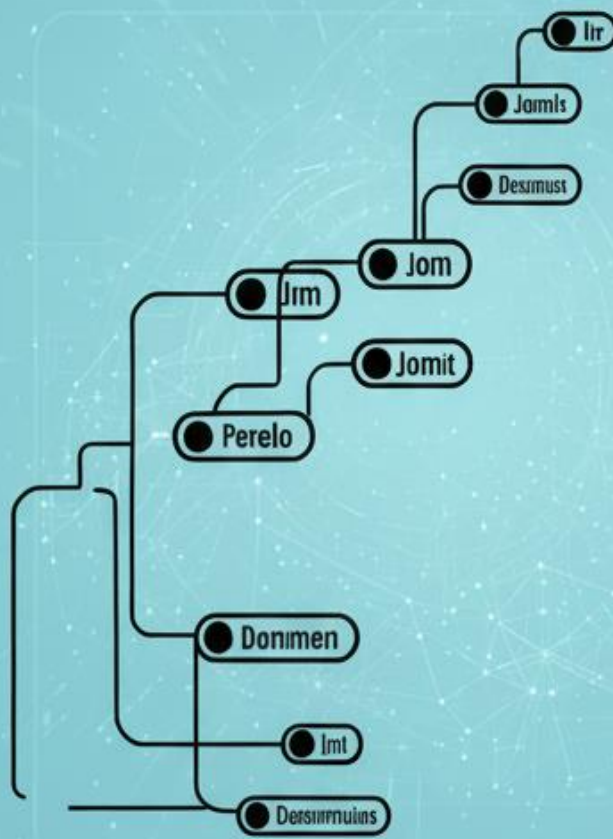
 Essa forma funciona, mas não é recomendada em projetos grandes, pois mistura HTML e JavaScript, dificultando manutenção.

Resumindo

- **Interno** → dentro da tag <script> no HTML.
- **Externo** → arquivo separado .js (boa prática).
- **Inline** → dentro de atributos HTML (evitar quando possível).

– Manipulando o DOM –

– Document Object Model



JS

FASE II



NO **MEIO** DO CAMINHO ..

FASE II – MANIPULANDO O DOM (*DOCUMENT OBJECT MODEL*)



DOM

Document Object Model

CONCEITO E ESTRUTURA

O que é o DOM?

- **DOM** significa **Document Object Model** (Modelo de Objeto de Documento).
- Ele é uma **representação em forma de árvore** do que está escrito em um arquivo **HTML** (ou XML).
- Quando você abre uma página no navegador, o HTML é **interpretado** e transformado nessa árvore DOM.

👉 Em resumo: **O DOM é a forma como o navegador "enxerga" e organiza a página, permitindo que o JavaScript interaja com ela** (alterando textos, imagens, cores, adicionando elementos, etc.).

🌳 Estrutura da árvore DOM

Pensa em uma árvore genealógica:

- **No topo** temos o document (o "pai de todos").
- Dentro dele, geralmente temos o html.
- O html tem dois "filhos principais": o head e o body.
- O body tem outros "filhos": h1, p, div, a, img, etc.
- Esses elementos também podem ter seus próprios filhos (um <div> pode ter um <p> dentro, por exemplo).

🔍 Exemplo prático

HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Minha Página</title>
  </head>
  <body>
    <h1>Olá, mundo!</h1>
    <p>Esse é um parágrafo.</p>
  </body>
</html>
```

Árvore DOM simplificada:

document

└─ html

 └─ head

```
|   └─ title → "Minha Página"
└─ body
    └─ h1 → "Olá, mundo!"
        └─ p → "Esse é um parágrafo."
```

🤖 Como o JavaScript usa o DOM?

Com JS, você pode acessar e modificar a árvore DOM. Por exemplo:

```
document.querySelector("h1").textContent = "Bem-vindo!";
```

Isso muda o texto do `<h1>` de **"Olá, mundo!"** para **"Bem-vindo!"**.

⚡ Resumindo:

- DOM é o **modelo em árvore** que o navegador cria a partir do HTML.
- Ele permite que **JavaScript interaja** com a página.
- Estrutura: **document > html > head/body > elementos > textos**.

ACESSANDO ELEMENTOS HTML VIA JAVASCRIPT NO DOM

Como acessar elementos HTML no DOM?

Existem vários jeitos, mas vou listar os principais meios:

1. **Por ID:** Se o elemento tem um atributo id, você pode pegar ele assim:

```
<p id="meuParagrafo">Olá, mundo!</p>
```

```
<script>
```

```
    let elemento = document.getElementById("meuParagrafo");
```

```
    console.log(elemento.innerText); // Mostra: Olá, mundo!
```

```
</script>
```

👉 Esse é rápido e direto.

2. **Por Classe:** Se o elemento tem class, você usa:

```
<p class="texto">Primeiro</p>
```

```
<p class="texto">Segundo</p>
```

```
<script>
```

```
    let elementos = document.getElementsByClassName("texto");
```

```
    console.log(elementos[0].innerText); // Primeiro
```

```
    console.log(elementos[1].innerText); // Segundo
```

```
</script>
```

👉 Retorna uma lista de elementos (parecida com array).

3. **Por Tag:** Seleciona todas as tags iguais

```
<h1>Título 1</h1>
```

```
<h1>Título 2</h1>
```

<script>

```
let titulos = document.getElementsByTagName("h1");  
console.log(titulos[0].innerText); // Título 1
```

</script>

4. Com querySelector: Pega o primeiro elemento que corresponder ao seletor CSS

<p class="texto">Olá!</p>

<script>

```
let elemento = document.querySelector(".texto"); // como no CSS (. para  
classe, # para id)
```

```
console.log(elemento.innerText); // Olá!
```

</script>

5. Com querySelectorAll: Pega todos os elementos que correspondem ao seletor CSS:

<p class="texto">Um</p>

<p class="texto">Dois</p>

<script>

```
let elementos = document.querySelectorAll(".texto");
```

```
elementos.forEach(e => console.log(e.innerText));
```

```
// Resultado: Um / Dois
```

</script>

OBSERVAÇÃO:

Quando eu disse “seletor CSS”, não quis dizer que você está pegando estilos (CSS), e sim que o jeito de escolher os elementos no querySelector e querySelectorAll é o mesmo jeito que você usaria no CSS para aplicar estilos.

Ou seja: você está pegando elementos HTML, só que usando a sintaxe do CSS para identificá-los.

Resumo rápido:

getElementById("id") → 1 elemento único.

getElementsByClassName("classe") → vários elementos.

getElementsByTagName("tag") → vários elementos.

querySelector("seletorCSS") → 1 elemento (primeiro que achar).

querySelectorAll("seletorCSS") → todos os elementos.

MANIPULANDO
ELEMENTOS HTML VIA
JAVASCRIPT
NO DOM

Perfeito 🙌 Agora que você já entendeu como acessar os elementos no DOM, vamos para a parte mais divertida: manipular esses elementos. Vou te explicar de forma bem simples e com exemplos.

🔗 1. Alterando conteúdos

Existem dois jeitos principais:

◆ **innerHTML**: Serve para mudar o conteúdo HTML interno de um elemento (pode incluir tags).

```
<p id="paragrafo">Texto original</p>
```

```
<script>
```

```
let p = document.getElementById("paragrafo");  
p.innerHTML = "<b>Texto em negrito</b>"; // agora o conteúdo tem HTML
```

```
</script>
```

◆ **textContent**: Serve para mudar apenas o texto (sem interpretar tags HTML).

```
<p id="paragrafo">Texto original</p>
```

```
<script>
```

```
let p = document.getElementById("paragrafo");  
p.textContent = "<b>Isso será exibido como texto</b>";
```

```
</script>
```

👉 **Resumo:**

innerHTML → pode inserir HTML.

textContent → só insere texto puro.

🚧 2. Alterando atributos

Usamos **setAttribute** e **getAttribute**.

```

```

```
<script>
```

```
let img = document.getElementById("foto");
```

```
// Alterar atributo
```

```
img.setAttribute("src", "nova.png");
```

```
// Pegar atributo
```

```
console.log(img.getAttribute("alt")); // mostra: foto
```

```
</script>
```

🚧 3. Alterando estilos (CSS pelo JS)

Usamos a propriedade **.style**

```
<p id="paragrafo">Olá, mundo!</p>
```

```
<script>
```

```
let p = document.getElementById("paragrafo");
```

```
p.style.color = "red"; // muda a cor do texto
```

```
p.style.fontSize = "20px"; // muda o tamanho da fonte
```

```
</script>
```

👉 **Só cuidado:** **.style** altera apenas inline styles (aqueles aplicados direto no elemento).

✦ 4. Manipulando classes (classList)

Esse é o mais usado no dia a dia.

```
<p id="paragrafo" class="texto">Olá!</p>
```

```
<script>
```

```
let p = document.getElementById("paragrafo");
```

```
p.classList.add("destaque"); // adiciona classe
```

```
p.classList.remove("texto"); // remove classe
```

```
p.classList.toggle("ativo"); // alterna: se tiver, remove; se não tiver, adiciona
```

```
</script>
```

✦ 5. Criando e removendo elementos

◆ Criando

```
<div id="container"></div>
```

```
<script>
```

```
let div = document.getElementById("container");
```

```
let novoParagrafo = document.createElement("p"); // cria <p>
```

```
novoParagrafo.textContent = "Sou um novo parágrafo!"; // adiciona texto
```

```
div.appendChild(novoParagrafo); // coloca dentro da div
```

```
</script>
```

◆ Removendo

```
<p id="paraRemover">Vou ser removido</p>
```

```
<script>
```

```
let p = document.getElementById("paraRemover");
```

```
p.remove(); // remove o elemento da página
```

```
</script>
```

Resumindo:

Conteúdo: innerHTML, textContent.

Atributos: setAttribute, getAttribute.

Estilos: element.style.propriedade.

Classes: classList.add, classList.remove, classList.toggle.

Criar/Remover: document.createElement, appendChild, remove().

EVENTOS NO DOM

CONCEITO E ESTRUTURA

1. O que são eventos na DOM?

Conceito:

Eventos são ações ou ocorrências que acontecem na página (como um clique, uma tecla pressionada ou um formulário enviado).

A DOM permite que você “escute” e “reaja” a essas ações.

Analogia:

Imagine a página como uma festa 🎉. Os eventos são coisas que acontecem na festa: alguém bate palmas (click), alguém fala seu nome (keydown), alguém toca a campainha (submit).

Exemplo:

```
<button id="btn">Clique aqui</button>
```

```
<script>
```

```
document.getElementById("btn").onclick = function() {  
    alert("Você clicou no botão!");  
}
```

```
</script>
```

🔗 2. O que é ouvir e responder a eventos?

Conceito:

Ouvir um evento é como ficar atento a uma ação.

Responder é executar um código quando essa ação acontece.

Analogia:

Um segurança numa festa 🕵️ está ouvindo para ver se alguém derruba um copo. Quando isso acontece, ele responde limpando o chão.

Exemplo:

```
<input type="text" id="campo" placeholder="Digite algo">
```

```
<script>
```

```
document.getElementById("campo").addEventListener("keydown", function() {  
    console.log("Você pressionou uma tecla!");  
});
```

```
</script>
```

📌 3. Como adicionar eventos (addEventListener)

Conceito:

addEventListener é a forma mais recomendada de associar um evento a um elemento. Ele permite que você adicione vários ouvintes para o mesmo evento.

Analogia:

É como ter várias pessoas ouvindo a mesma música 🎵: cada um pode reagir de um jeito diferente (dançar, cantar, bater palma).

Exemplo:

```
<button id="btn">Clique</button>
```

```
<script>
```

```
let botao = document.getElementById("btn");
```

```
botao.addEventListener("click", () => {  
  alert("Primeira reação!");  
});
```

```
botao.addEventListener("click", () => {  
  alert("Segunda reação!");  
});
```

```
</script>
```

📌 4. O objeto event

Conceito:

Toda vez que um evento acontece, o navegador cria um objeto chamado event. Esse objeto guarda informações sobre o evento, como:

- ✓ Qual tecla foi pressionada
- ✓ Onde o mouse estava
- ✓ Qual elemento foi clicado

Analogia:

É como uma ata de ocorrência 📝 de um policial: toda vez que algo acontece, ele registra detalhes do ocorrido.

Exemplo:

```
<input type="text" id="campo" placeholder="Digite algo">
```

```
<script>
```

```
document.getElementById("campo").addEventListener("keydown",  
function(event) {
```

```
    console.log("Tecla pressionada: " + event.key);
```

```
});
```

```
</script>
```

🔥 5. Propagação de eventos (Event Bubbling e Capturing)

Conceito:

Quando você clica em um elemento dentro de outro, o evento pode se propagar pela árvore DOM em duas fases:

Capturing (captura): o evento vai do topo da árvore (document) até o elemento clicado.

Bubbling (borbulhamento): depois, o evento sobe de volta, do elemento clicado até o topo.

Por padrão, os eventos usam **bubbling**.

Analogia:

Imagine jogar uma bolinha numa piscina 🏊.

Capturing é a bolinha descendo até o fundo.

Bubbling são as bolhas subindo de volta à superfície.

Exemplo:

```
<div id="pai" style="padding:20px; background:lightblue;">
```

```
  Pai
```

```
    <button id="filho">Filho</button>
```

```
</div>
```

<script>

```
document.getElementById("pai").addEventListener("click", () => {  
  alert("Cliquei no PAI (bubbling)");  
});
```

```
document.getElementById("filho").addEventListener("click", () => {  
  alert("Cliquei no FILHO");  
});
```

// Exemplo de capturing

```
document.getElementById("pai").addEventListener("click", () => {  
  alert("Cliquei no PAI (capturing)");  
}, true);
```

</script>

 **Resumo rápido:**

Eventos: coisas que acontecem na página (clique, tecla, etc.).

Ouvir e responder: ficar atento a algo e executar uma ação quando acontece.

addEventListener: forma moderna de adicionar reações a eventos.

Objeto event: informações detalhadas sobre o que ocorreu.

Propagação: o evento pode ir descendo (capturing) e depois subindo (bubbling) na árvore DOM.

JavaScript Assíncrono e ES6+

FASE III

```
const getPost = () => fetch('https://jsonplaceholder')
```

```
function App() {  
  const [post, updatePosts] = useState(null)
```

```
  useEffect(() => {  
    (async () => {  
      updatePosts(await getPost())  
    })();  
  }, []);
```

```
  if (!post) {  
    return <div>Loading... </div>  
  }
```

```
  return (  
    <div>  
      <h2>Hello World</h2>  
    </div>  
  )  
}
```

ESTAMOS **QUASE** LÁ ..

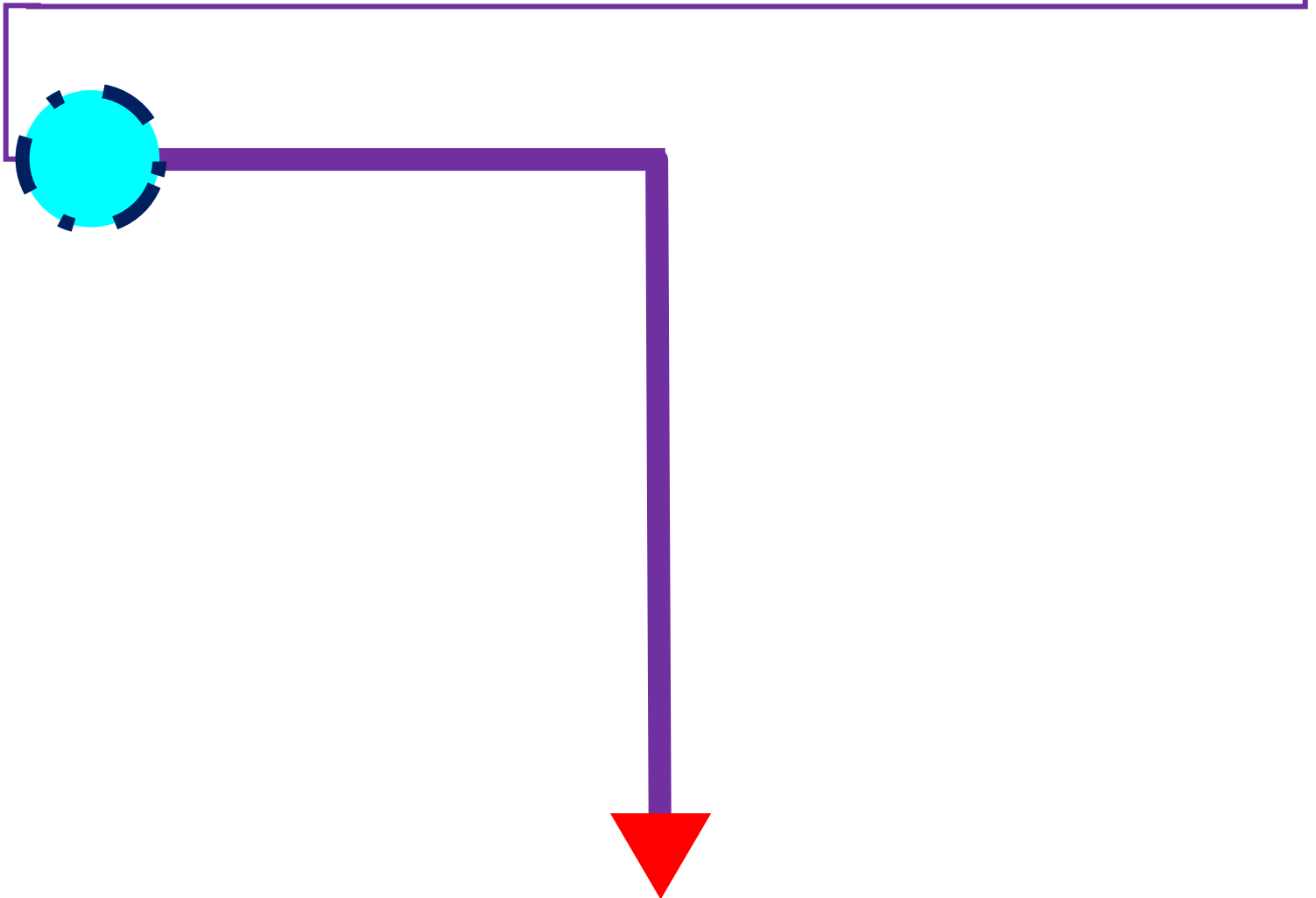
FASE III – JAVASCRIPT ASSÍNCRONO E ES6+

1º

JAVASCRIPT
ASSÍNCRONO -
CONCEITO E
ESTRUTURA

2º

NOVAS FEATURES
DO
ECMAScript 6+



PROSSIGA!

JAVASCRIPT ASSÍNCRONO

CONCEITO E ESTRUTURA

O que é "assíncrono" em JavaScript?

👉 JavaScript normalmente executa as tarefas linha por linha (de forma síncrona).

Mas algumas tarefas demoram (ex: buscar dados na internet, ler arquivos, esperar tempo).

Se ele fosse esperar cada uma terminar, a página “travaria”.

➡ Então o assíncrono permite que o JS continue rodando outras coisas enquanto espera a tarefa demorada.

Analogia:

Imagine que você está cozinhando macarrão.

Você coloca a água no fogo (demora).

Enquanto espera a água ferver, você corta os legumes.

Isso é programação assíncrona: você não fica parado esperando a água ferver, você faz outras coisas.

◆ Callbacks

Um callback é simplesmente uma função passada como argumento para outra função, que será chamada depois de um tempo ou quando algo terminar.

Analogia:

Pense num restaurante: você pede uma pizza e deixa seu número de telefone (callback).

Quando a pizza ficar pronta, eles ligam para você.

Você não fica esperando parado, vai fazer outras coisas.

Exemplo com setTimeout:

```
function pedirPizza(callback) {  
  console.log("Pedido feito...");  
  setTimeout(() => {  
    console.log("Pizza pronta!");  
    callback();  
  }, 2000); // espera 2 segundos  
}  
  
pedirPizza(() => {  
  console.log("Comendo a pizza 🍕");  
});
```

Problema: Callback Hell

Quando temos muitas coisas em sequência (uma depende da outra), os callbacks ficam um dentro do outro, criando um “código em pirâmide”, difícil de ler e manter.

```
buscarUsuario(1, (usuario) => {  
  buscarPedidos(usuario, (pedidos) => {  
    buscarDetalhes(pedidos, (detalhes) => {  
      console.log(detalhes);  
    });  
  });  
});
```

👉 Isso é o famoso "Callback Hell".

◆ Promises

As Promises surgiram para resolver o problema dos callbacks aninhados.

Elas representam algo que vai acontecer no futuro:

- 🟡 Pending (pendente)
- 🟢 Fulfilled (resolvida com sucesso)
- 🔴 Rejected (falhou)

Analogia:

Uma Promise é como um bilhete de retirada da sua pizza.

- ✓ Enquanto está no forno = Pending
- ✓ Pizza pronta = Fulfilled
- ✓ Queimou/errou pedido = Rejected

Exemplo:

```
let promessa = new Promise((resolve, reject) => {  
  let sucesso = true;  
  setTimeout(() => {  
    if (sucesso) resolve("Pizza entregue 🍕");  
    else reject("Deu ruim, pizza queimada 😞");  
  }, 2000);  
});
```

promessa

```
.then((resultado) => console.log("Sucesso:", resultado)) // se deu certo
.catch((erro) => console.log("Erro:", erro))           // se deu errado
.finally(() => console.log("Processo finalizado"));
```

◆ Async/Await

O async/await é apenas uma forma mais simples de escrever Promises, deixando o código com cara de “síncrono”, mas ainda é assíncrono.

Analogia:

É como usar WhatsApp para pedir a pizza.

Você manda a mensagem (async), e espera sentado até receber a resposta (await).

O código fica mais “limpo”.

Exemplo:

```
async function pedirPizza() {
  try {
    let resultado = await promessa; // espera a Promise terminar
    console.log("Sucesso:", resultado);
  } catch (erro) {
    console.log("Erro:", erro);
  } finally {
    console.log("Processo finalizado");
  }
}

pedirPizza();
```

◆ Fetch API

O Fetch API é usado para fazer requisições HTTP (ex: buscar dados de uma API, enviar formulários, etc).

Ele retorna uma Promise.

Analogia:

É como pedir informações numa central de atendimento:

- 📞 Você faz a requisição (ligação).
- 📞 Espera a resposta (Promise).
- 📞 Depois usa os dados recebidos.
- 📞 Exemplo (pegando dados de uma API pública):

// Usando Promises

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
  .then(response => response.json()) // converte para JSON  
  .then(data => console.log("Dados:", data))  
  .catch(error => console.log("Erro:", error));
```

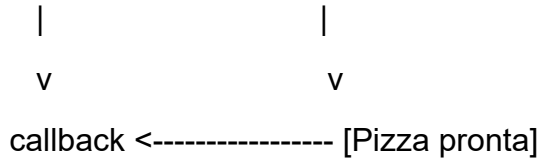
// Usando Async/Await

```
async function pegarDados() {  
  try {  
    let response = await fetch("https://jsonplaceholder.typicode.com/posts/1");  
    let data = await response.json();  
    console.log("Dados:", data);  
  } catch (error) {  
    console.log("Erro:", error);  
  }  
}  
  
pegarDados();
```

Vou te mostrar um fluxo visual passo a passo comparando cada técnica (Callback → Promise → Async/Await → Fetch).

◆ 1. Callback (o jeito mais antigo)

[Você pede a pizza] ----> [Pizzaria recebe o pedido]

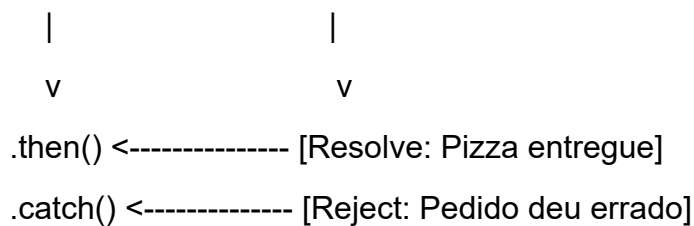


👉 Você deixa um telefone (callback), e quando a pizza fica pronta, eles te ligam.

⚠️ Se tiver que pedir pizza + bebida + sobremesa → um callback dentro do outro = callback hell.

◆ 2. Promise

[Você pede a pizza] ----> [Recebe um bilhete (Promise)]



👉 Você ganha um bilhete (Promise) no momento do pedido.

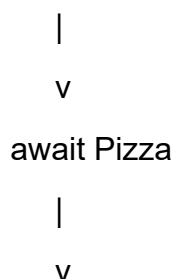
Se der certo → .then()

Se der errado → .catch()

Em qualquer caso → .finally()

◆ 3. Async / Await

[Você pede a pizza]



[Pizza entregue 🍕]



O c

[JS



// C

faz

// P

faz

II A

asy

```
try {
```

```
let resposta = await fazerPedidoPizzaPromise();
```

```
console.log("Comendo pizza (async/await):", resposta);
```

```
} catch (erro) {
```

```
console.log("Erro:", erro);
```

}

}

```
comerPizza();
```

// Fetch API

```
async function buscarDados() {  
  let response = await fetch("https://jsonplaceholder.typicode.com/posts/1");  
  let dados = await response.json();  
  console.log("Dados da API:", dados);  
}  
buscarDados();
```

👉 Assim dá pra ver a evolução: Callback → Promise → Async/Await → Fetch.

✅ Resumo:

Callbacks → função chamada depois (mas pode virar bagunça = callback hell).

Promises → resolvem esse problema, com `.then()` / `.catch()` / `.finally()`.

Async/Await → deixam Promises mais fáceis de ler/escrever.

Fetch API → usada para buscar dados de APIs (trabalha com Promises).

NOVAS FEATURES DO **ECMASCRIPT 6+**

2015 E POSTERIORES

1. Desestruturação de arrays e objetos

Conceito:

É um jeito de “desmontar” arrays ou objetos em variáveis individuais de forma rápida.

Analogia:

Imagine que você comprou uma caixa de frutas. Em vez de abrir e pegar cada fruta manualmente, você já tira maçã, banana e laranja direto da caixa para a mesa.

Exemplo:

// Array

```
const frutas = ["maçã", "banana", "laranja"];  
const [f1, f2, f3] = frutas;  
console.log(f1, f2, f3); // maçã banana laranja
```

// Objeto

```
const pessoa = { nome: "Ana", idade: 25 };  
const { nome, idade } = pessoa;  
console.log(nome, idade); // Ana 25
```

2. Template Literals (crases e variáveis dentro de strings)

Conceito:

São strings que permitem usar variáveis e expressões dentro de crases (` `), sem precisar concatenar com +.

Analogia:

É como escrever uma carta usando campos automáticos: “Olá, [nome], você tem [idade] anos”.

Exemplo:

```
const nome = "Robson";  
const idade = 27;  
  
console.log(`Olá, meu nome é ${nome} e eu tenho ${idade} anos.`);  
// Olá, meu nome é Robson e eu tenho 27 anos.
```

3. Operador Spread (...)

Conceito:

O Spread espalha os elementos de um array ou objeto.

Analogia:

Pensa em abrir um baralho na mesa: em vez de entregar o baralho fechado, você espalha todas as cartas.

Exemplo:

```
const numeros = [1, 2, 3];  
const novosNumeros = [...numeros, 4, 5];  
console.log(novosNumeros); // [1, 2, 3, 4, 5]
```

4. Operador Rest (...)

Conceito:

O Rest junta vários elementos em uma só variável (faz o oposto do Spread).

Analogia:

É como pegar vários brinquedos espalhados e guardá-los numa caixa só.

Exemplo:

```
function soma(...numeros) {  
  return numeros.reduce((total, n) => total + n, 0);  
}  
console.log(soma(1, 2, 3, 4)); // 10
```

5. Módulos ES6 (import/export)

Conceito:

Permitem dividir o código em arquivos e depois importar/exportar funções, classes ou variáveis.

Analogia:

É como uma loja de peças de carro: você compra só a peça que precisa, não o carro inteiro.

Exemplo:

// arquivo math.js

```
export function soma(a, b) {  
  return a + b;  
}
```

// arquivo main.js

```
import { soma } from "./math.js";  
console.log(soma(2, 3)); // 5
```

6. Classes (açúcar sintático para protótipos)

Conceito:

Uma forma mais bonita e organizada de criar objetos e herança em JavaScript. Por baixo, continua sendo protótipos, mas a sintaxe é mais clara.

Analogia:

É como montar uma forma de bolo: você cria a forma (classe) e dela saem vários bolos (objetos).

Exemplo:

```
class Animal {  
  constructor(nome) {  
    this.nome = nome;  
  }  
  falar() {  
    console.log(`${this.nome} faz um som.`);  
  }  
}
```

```
class Cachorro extends Animal {  
  falar() {  
    console.log(`${this.nome} late.`);  
  }  
}
```

```
const dog = new Cachorro("Rex");  
dog.falar(); // Rex late.
```

7. Map

Conceito:

É uma estrutura parecida com objeto, mas feita para mapear chave → valor. Diferente de objetos, as chaves podem ser de qualquer tipo (número, objeto, etc).

Analogia:

É como uma lista telefônica, onde cada pessoa (chave) tem um número (valor).

Exemplo:

```
const mapa = new Map();  
mapa.set("nome", "Carlos");  
mapa.set("idade", 30);  
  
console.log(mapa.get("nome")); // Carlos  
console.log(mapa.has("idade")); // true
```

8. Set

Conceito:

É uma estrutura que guarda valores únicos, sem repetir.

Analogia:

É como uma caixa de figurinhas: se você tentar colocar uma figurinha repetida, ela não entra.

Exemplo:

```
const numeros = new Set([1, 2, 2, 3, 4, 4]);  
console.log(numeros); // Set {1, 2, 3, 4}
```

👉 Essas são algumas das principais features do ES6+, que deixaram o JavaScript mais poderoso e mais fácil de escrever.

FERRAMENTAS MODERNAS E ECOSSISTEMA

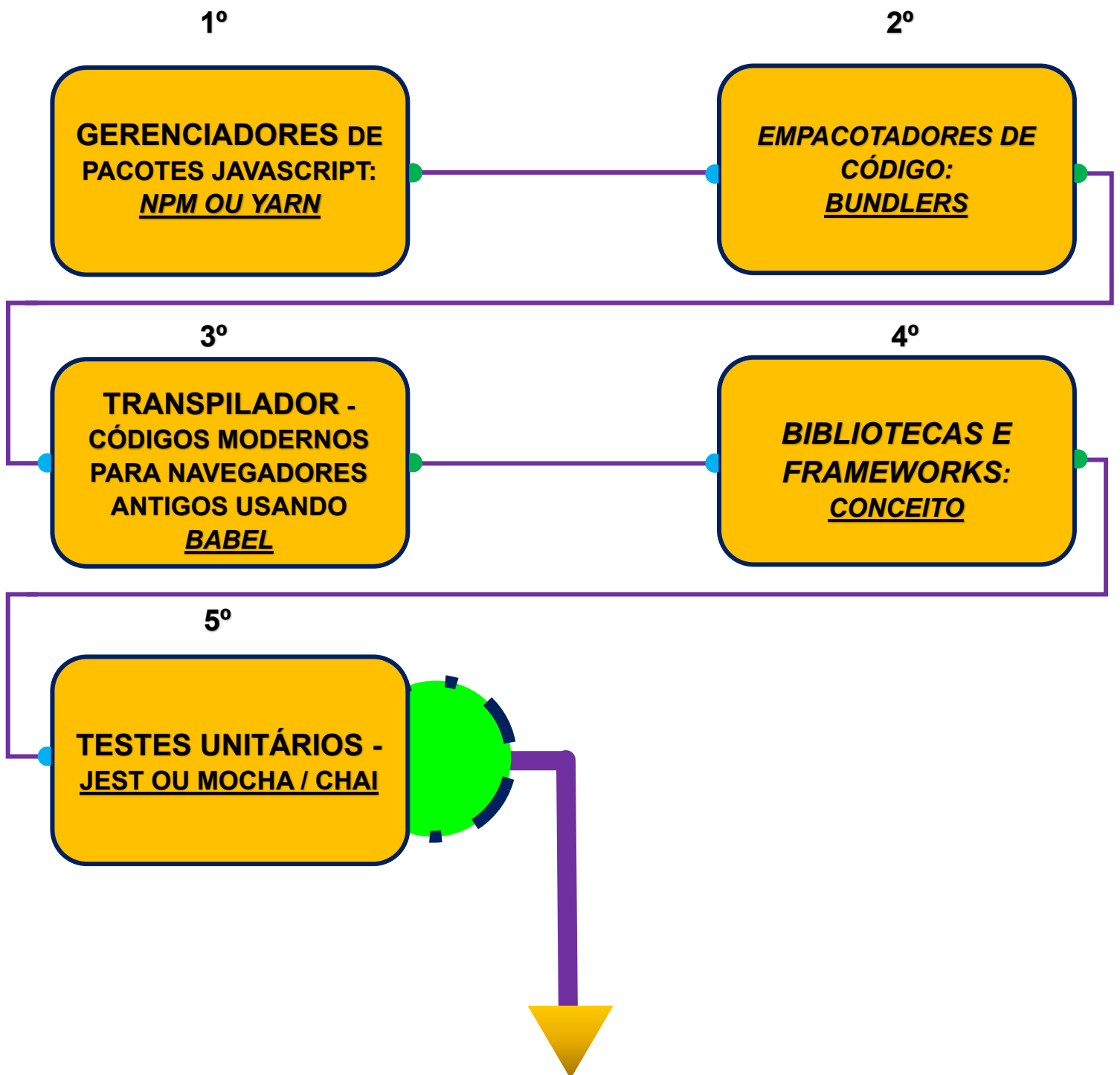


FASE IV



DESTINO **FINAL**

FASE IV – FERRAMENTAS MODERNAS E ECOSSISTEMA



EM FRENTE!

GERENCIADORES DE PACOTES JAVASCRIPT

NPM ou YARN

Vou te explicar de forma simples como funciona o NPM e o Yarn, como instalar e começar a usar, e como eles ajudam a gerenciar dependências.



O que são NPM e Yarn?

NPM (Node Package Manager) e Yarn são gerenciadores de pacotes para projetos em JavaScript/Node.js.

Eles servem para instalar, atualizar e remover bibliotecas (chamadas de dependências) que seu projeto precisa.

Analogia:

Pense no seu projeto como uma casa.

O NPM/Yarn é tipo uma loja de materiais de construção.

As dependências (React, Express, Axios, etc.) são os materiais (tijolos, cimento, portas).

O package.json é a lista de compras, onde ficam registradas todas as dependências que você usou.



Como instalar

Instalar Node.js

O NPM já vem junto com o Node.js.

Baixe em: <https://nodejs.org>

Para conferir se deu certo:

node -v # mostra versão do Node

npm -v # mostra versão do npm

Instalar Yarn (opcional, alternativa ao npm)

Se quiser usar o Yarn:

`npm install -g yarn`

Depois confira:

yarn -v

Iniciando um projeto

No terminal, dentro da pasta do seu projeto:

```
npm init -y # cria o package.json automaticamente
```

ou

```
yarn init -y
```

 Isso cria o **package.json**, que guarda informações do projeto e suas dependências.

Instalando dependências

Existem dois tipos principais:

Dependências normais → usadas no projeto em produção

Dependências de desenvolvimento → usadas só durante o desenvolvimento (ex.: testes, linters)

Instalar uma dependência

```
npm install react # npm
```

```
yarn add react # yarn
```

Instalar dependência de desenvolvimento

```
npm install jest --save-dev # npm
```

```
yarn add jest --dev # yarn
```

Outras dependências

Instalar todas dependências listadas no package.json

```
npm install
```

```
yarn install
```

Remover uma dependência

```
npm uninstall axios
```

```
yarn remove axios
```

Atualizar dependências

npm update

yarn upgrade

🔧 Rodando scripts

No **package.json**, você pode definir scripts, por exemplo:

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
}
```

E rodar:

npm run start

yarn start

✅ Resumindo:

NPM e Yarn são gerenciadores de pacotes.

Usam o **package.json** para controlar dependências.

Principais comandos: init, install, uninstall, update, run.

EMPACOTADORES DE CÓDIGO

BUNDLERS

Vamos falar sobre Bundlers (empacotadores de código), como funcionam, para que servem, quais existem e um exemplo de configuração básica.

O que é um Bundler?

Um Bundler (empacotador) é uma ferramenta que pega todos os arquivos do seu projeto (JavaScript, CSS, imagens, etc.) e empacota em arquivos otimizados para o navegador.

Isso é necessário porque o navegador não entende diretamente módulos ES6, imports complexos ou vários arquivos espalhados.

Analogia:

Imagine que você vai viajar .

Você tem roupas, tênis, documentos, carregador... tudo espalhado.

O Bundler funciona como uma mala de viagem: ele junta tudo, organiza e comprime para você carregar só uma bagagem pronta.

Assim o navegador só recebe os arquivos finais otimizados.

Principais Bundlers

Webpack → O mais popular e poderoso, mas um pouco mais complexo.

Parcel → Configuração quase zero, fácil de começar.

Rollup → Mais usado em bibliotecas, gera pacotes menores e otimizados.

O que um Bundler faz?

Junta arquivos → combina vários .js em um único bundle.

Transpila código → usa o Babel para transformar código moderno (ES6+) em algo que funciona em navegadores antigos.

Processa outros arquivos → pode importar CSS, imagens, JSON etc.

Otimiza → minifica (remove espaços), comprime e melhora a performance.

🔧 Exemplo de Configuração Básica (Webpack)

Suponha que você tem um projeto com index.js e quer empacotar.

1. Instalar dependências

```
npm init -y
```

```
npm install webpack webpack-cli --save-dev
```

2. Estrutura de pastas

meu-projeto/

```
|— src/
|   └─ index.js
|— dist/
|   └─ index.html
|— package.json
└─ webpack.config.js
```

3. Criar arquivo webpack.config.js

```
const path = require('path');
```

```
module.exports = {
  entry: './src/index.js',    // Arquivo inicial
  output: {
    filename: 'bundle.js',    // Arquivo final empacotado
    path: path.resolve(__dirname, 'dist')
  },
  mode: 'development'        // ou "production"
};
```

4. Ajustar package.json

```
"scripts": {
  "build": "webpack"
}
```

5. Rodar

npm run build

👉 Isso vai gerar **dist/bundle.js** que você importa no seu index.html.

⚡ Parcel (mais simples)

Se usar o Parcel, nem precisa de config no início:

npm install -g parcel-bundler

parcel src/index.html

Ele já entende imports de JS, CSS e imagens automaticamente.

✅ **Resumindo:**

Bundlers organizam e otimizam seu código para rodar no navegador.

Webpack → mais completo, mas exige configuração.

Parcel → quase sem configuração.

Rollup → ótimo para bibliotecas.

TRANSPILADOR
CÓDIGOS MODERNOS PARA
NAVEGADORES ANTIGOS
USANDO BABEL

O que é um Transpilador?

Um transpilador é uma ferramenta que converte código moderno (ES6+, TypeScript, JSX, etc.) em uma versão mais antiga de JavaScript que todos os navegadores entendem.

O mais famoso é o Babel.

Analogia:

Pense que você escreve em português atual com gírias e expressões modernas.

Mas seu avô só entende português antigo.

O Babel é como um tradutor, que pega seu português moderno e reescreve de uma forma que seu avô entenda.

Por que usar Babel?

Nem todos os navegadores suportam todas as features novas do JavaScript.

Exemplo: let, const, arrow functions, async/await, etc.

O Babel garante que seu código funcione em qualquer navegador, mesmo os mais antigos.

Como funciona o Babel?

Você escreve código moderno (ES6+):

```
const soma = (a, b) => a + b;
```

O Babel converte para uma versão antiga:

```
var soma = function(a, b) {  
  return a + b;  
};
```

O navegador mais antigo consegue rodar sem problemas.

Como instalar e usar

Criar projeto:

```
npm init -y
```

Instalar Babel:

```
npm install @babel/core @babel/cli @babel/preset-env --save-dev
```

Criar arquivo .babelrc (configuração):

```
{  
  "presets": ["@babel/preset-env"]  
}
```

Rodar Babel no código:

```
npx babel src --out-dir dist
```

 Isso pega tudo em src/ e converte para código compatível em dist/.

 Babel + Bundlers

Normalmente, o Babel é usado junto com bundlers (como Webpack, Parcel, Vite).

O bundler organiza os arquivos e o Babel traduz para navegadores antigos.

É tipo uma dupla de tradutor + organizador.

Resumindo:

Babel é um tradutor que converte código moderno para versões antigas de JavaScript.

Garante que seu app rode em qualquer navegador.

É muito usado em conjunto com Webpack, Parcel, Vite.

BIBLIOTECAS E FRAMEWORKS

CONCEITO

Vamos falar de Frameworks e Bibliotecas no mundo do JavaScript, e depois explicar como funcionam React, Vue.js e Angular, com analogias para ficar fácil.

📌 O que são Bibliotecas e Frameworks?

Biblioteca: é um conjunto de ferramentas que você pode usar quando precisar.

👉 Você escolhe quando e como usá-la.

Framework: é como uma estrutura pronta para construir seu projeto.

👉 Ele já dita como você deve organizar seu código, e você precisa se adaptar às regras dele.

👉 Analogia simples

Imagine que você vai cozinhar: 🔍

Biblioteca → É como um livro de receitas.

Você escolhe a receita que quer usar e aplica no seu prato.

(Ex.: **React** → você pega os “componentes” e usa quando quiser).

Framework → É como uma caixa de refeição pronta.

Vem com os ingredientes, instruções e até a ordem de preparo.

Você pode personalizar, mas precisa seguir o padrão.

(Ex.: **Angular** → ele já diz como organizar rotas, serviços, componentes).

📌 Exemplos práticos

■ React (Biblioteca)

Criado pelo Facebook.

É uma biblioteca focada só na camada de interface (UI).

Baseado em componentes (pequenas peças de código que representam partes da tela).

👉 Analogia:

O React é como um conjunto de peças de LEGO.

Você pode montar um carrinho, uma casa ou um castelo, mas quem decide a forma é você.

■ Vue.js (Framework/Biblioteca híbrido)

Criado por Evan You.

É mais flexível: pode ser usado só como biblioteca (como React) ou como framework completo.

Simples de aprender e muito usado para projetos rápidos.

👉 Analogia:

O Vue é como um kit de cozinha modular.

Você pode usar só uma panela (biblioteca) ou montar a cozinha inteira (framework).

■ Angular (Framework)

Criado pelo Google.

É um framework completo para construir aplicações grandes.

Já vem com tudo: rotas, formulários, injeção de dependência, testes, etc.

👉 Analogia:

O Angular é como uma cozinha industrial completa.

Já tem todas as máquinas e utensílios no lugar, mas você precisa aprender a usá-los da forma que o manual manda.

📌 Resumindo

Biblioteca → você controla o fluxo e usa só o que precisa (ex.: React).

Framework → ele controla o fluxo e te obriga a seguir um padrão (ex.: Angular).

Vue.js → fica no meio termo, pode ser leve como biblioteca ou completo como framework.

TESTES UNITÁRIOS

JEST OU MOCHA/CHAI

Vamos falar sobre Testes Unitários, para que servem, como funcionam, e exemplos de ferramentas como Jest e Mocha/Chai.

📌 O que são Testes Unitários?

Teste unitário é um tipo de teste que verifica pequenas partes do código (as “unidades”), como funções ou métodos, para garantir que estão funcionando corretamente.

A ideia é testar cada peça isoladamente, sem depender de outras partes do sistema.

👉 Analogia simples

Imagine que você está montando um carro 🚗.

Antes de montar tudo, você testa o freio, o motor, o câmbio individualmente.

Isso garante que cada peça funciona sozinha, e depois, quando montar o carro todo, as chances de dar problema são muito menores.

Os testes unitários fazem exatamente isso no software: **testam cada função ou componente separadamente.**

📌 Pra que servem?

Evitar bugs → você descobre erros cedo.

Facilitar manutenção → se mudar algo no código, o teste avisa se quebrou algo.

Dar confiança → você pode atualizar o sistema sem medo de quebrar outra parte.

⚙️ Como funcionam na prática?

Você escreve uma função no código:

```
function soma(a, b) {  
  return a + b;  
}
```

Depois escreve um teste unitário para ela:

```
test('soma 2 + 2 deve ser 4', () => {  
  expect(soma(2, 2)).toBe(4);  
});
```

O framework de testes (como **Jest**) roda e confirma se o resultado esperado bate com o resultado real.

Se deu certo → 

Se deu errado →  mostra o erro.

Ferramentas de Testes

Jest

Criado pelo Facebook.

Muito usado com React, mas serve para qualquer JS.

Simples, já vem com tudo configurado.

Exemplo:

```
test('multiplica 3 por 3 deve ser 9', () => {  
  expect(3 * 3).toBe(9);  
});
```

Mocha + Chai

Mocha → o motor que roda os testes.

Chai → biblioteca de asserções (formas de verificar se deu certo).

Mais flexível, mas precisa configurar.

Exemplo:

```
const { expect } = require('chai');  
  
describe('Teste de soma', () => {  
  it('2 + 3 deve ser 5', () => {  
    expect(2 + 3).to.equal(5);  
  });  
});
```

Resumindo

Teste Unitário → testa partes pequenas do código de forma isolada.

Serve para evitar erros, dar confiança e facilitar manutenção.

Jest → fácil de usar, tudo em um só pacote.

Mocha + Chai → mais flexível, mas exige configuração.

CONGRATULATIONS!

VOCÊ AGORA

É UM

**DESENVOLVEDOR
JAVASCRIPT**